Buone Pratiche di Programmazione e Applicazioni Reali del Linguaggio Python

A cura di Marco Pierleoni

Le caratteristiche di Python

Python è un linguaggio di programmazione ad alto livello, orientato a oggetti, e utilizzato in diversi ambiti tra cui:

- il Web Development
- Data Science e Analisi
- Intelligenza artificiale e Machine Learning
- Automazioni e Scripting

Python, difatti, è molto apprezzato per le sue caratteristiche che lo hanno reso un linguaggio di programmazione molto utilizzato sia in diversi ambiti, sia da programmatori esperti che non.
Le caratteristiche di python sono:

1. **Sintassi semplice e leggibile**
2. **Versatilità**
3. **Ampia libreria standard**
4. **Ricco ecosistema di librerie e framework**
5. **Community attiva e supporto**
6. **Portabilità**
7. **Interprete e sviluppo rapido**
8. **Supporto per il testing**
9. **Integrazione con altri linguaggi**
10. **Adatto a diversi livelli di competenza**

Breve Storia di Python

Python fu creato all'inizio degli anni novanta dall'informatico olandese Guido van Rossum.

Il nome fu scelto per la passione di G. van Rossum per i Monty Python e per la loro serie televisiva *Monty Python's Flying Circus*

[1]

Le buone pratiche di programmazione per Python

Per quanto riguarda Python, esistono delle linee guida, o per meglio dire, delle buone pratiche di programmazione che sono finalizzate a rendere il codice:

- **Leggibile**
- **facile da gestire**
- **efficiente**

Seguendo le convenzioni scritte nelle **PEP** (**Python Enhancement Proposal)** si facilita la collaborazione tra sviluppatori e si riduce il rischio di errori.

Le buone pratiche di programmazione per Python: le PEP

Esistono centinaia di PEP ognuna con un numero univoco che le differenzia tra loro.
Propongono nuove funzionalità, miglioramenti o modifiche per il linguaggio di Python, si dividono principalmente in:

1. **PEP di standardizzazione:**

Propongono modifiche tecniche (es: nuove funzionalità e modifiche al linguaggio)

1. **PEP informative:**
   Forniscono linee guida o informazioni utili (es: PEP 8 per lo stile del codice)
2. **PEP di processo:**
   Forniscono regole per il processo di sviluppo del linguaggio

Elenco delle buone pratiche di programmazione per Python

Come detto prima, queste buone pratiche di programmazione in python permettono di rendere il codice leggibile ed efficiente.

Alcune di queste pratiche sono:

1. **Naming (Nomenclatura)**
2. **Indentation**
3. **Equality checking**

Elenco delle buone pratiche di programmazione per Python: Naming

La nomenclatura è fondamentale per rendere il codice leggibile e organizzato, ma anche per evitare conflitti di nomi nel codice.

Le regole principali sono:

1. **Variabili, funzioni, metodi, pacchetti e moduli:**
   è bene utilizzare nomi in minuscolo separati da underscore (es: *calculate_sum).*
2. **Classi ed eccezioni:**
   si utilizza il formato CamelCase (es: *MyClass).*
3. **Metodi protetti:**
   in questo caso si deve utilizzare l'underscore all'inizio (es:*_protected_method)*
4. **Metodi privati:**
   si deve aggiungere due undescore all'inizio (es:*__procteted_method)*
5. **Costanti:**
   si deve scriverle in maiuscolo, separate da underscore (es: *MAX_VALUE)*

Linee Guida generali per la nomenclatura

- Evitare variabili con una sola lettera (es: *l, o, i,* etc.) a meno che non vengano usate in blocchi brevi dove il significato è chiaro dal contesto.

Esempio:
for e in elements:
e.mutate()

- Evitare etichette ridondanti.

Esempio:
import audio
core = audio.Core()
controller = audio.Controller()

- Usare una notazione inversa per maggiore chiarezza.

Esempio:
elements = ...
elements_active = ...
elements_defunct = ...

Elenco delle buone pratiche di programmazione per Python: Indentation

L’indentazione deve essere coerente poiché è

1. **essenziale per definire blocchi di codice**
2. **aiuta a capire la struttura del programma** (ad esempio: blocchi all’interno dei cicli, funzioni o condizioni)
3. **facilita la lettura del codice e la collaborazione tra gli sviluppatori**.

Elenco delle buone pratiche di programmazione per Python: Equality checking

I confronti di uguaglianza si riferiscono al modo in cui confrontiamo i valori.

Per questo è bene evitare confronti espliciti con ***True, False o None.***

Yes

```python
if attr:
    print('True!')

if attr is True:
    print('True!')

if not attr:
    print('False!')

if attr is None:
    print('None')
```

No

```python
if attr == True:
    print('True!')

if attr == False:
    print('False!')

if attr == None:
    print('None')
```

[2]

Applicazioni reali del linguaggio Python

Come detto alla slide n.2, python viene implementato in moltissimi ambiti tra cui quello delle automazioni e scripting.

- **Automazione (Automation):**
  si riferisce all'uso di script per eseguire compiti ripetitivi o automatizzati senza l'intervento dell'utente
- **Scripting:**
  si riferisce alla scrittura di piccoli programmi (script) per eseguire compiti specifici che possono anche essere usati per l'automazione.

Implementazioni di automazioni e scripting usando Python

1. **Automazione :**
   - Gestione e spostamento di file in cartelle (utile ad esempio per archiviare documenti o immagini).
   - Scraping di dati da siti web per raccogliere informazioni in tempo reale.
   - Automazione di test software per verificare che il codice funzioni correttamente su più dispositivi.
2. **Scripting:**
   - Scrittura di script per analizzare file di log
   - Creazione di script per interagire con API (es: per raccogliere o inviare i dati).
   - Batch processing di dati (ad esempio modificare grandi quantità di file CSV).

Vantaggi e svantaggi nell'automazione e scripting con Python

Vantaggi	Svantaggi
Semplicità e leggibilità del codice	Prestazioni inferiori rispetto ad altri linguaggi
Supporto per attività di automazione comuni	Problemi con l'uso intensivo della CPU e della memoria
Elevata produttività e rapidità nello sviluppo	Gestione dei thread
Automazione di operazioni su sistemi operativi differenti	Distribuzione e creazione di eseguibili stand-alone
Facilità di integrazione con altre tecnologia e API	Gestione delle dipendenze
Gestione dei task schedulati e periodici	Compilazione e ottimizzazione limitate
Documentazione e supporto della community	

References

- T. Cuzick, 2024, **Beginner's Guide To Python Automation Scripts (With Code Examples).** Zerotomastery.io.https://zerotomastery.io/blog/python-automation-scripts-beginners-guide/#What-are-automation-scripts
- Ruimaranhao/python-guide-ist.md, 2024, **A Guide of Best Practices for Python.** gist.github.com. https://gist.github.com/ruimaranhao/4e18cbe3dad6f68040c32ed6709090a3
- J. Canales Luna, 2024, **6 Python Best Practices for Better Code.** Datacamp.com. https://www.datacamp.com/blog/python-best-practices-for-better-code
- The PEP Editors, 2000, **PEP 0 – Index of Python Enhancement Proposals (PEPs).** Peps.python.org. https://peps.python.org/#
- Geeksforgeeks, 2024, **Python Features.** Geeksforgeeks.org. https://www.geeksforgeeks.org/python-features/
- Tutorialspoints,2022, **What is PEP for Python?**. Tutorialpoints.com. https://www.tutorialspoint.com/what-is-pep-for-python
- A. Verma,2024, **Decoding Python Automation and Scripting.** Pickl.ai. https://www.pickl.ai/blog/python-automation-scripting/
- H. Sharer, 2023, **Python Scripting for Automation: A Step-by-Step Guide.** Lihpao.com. https://www.lihpao.com/how-to-write-python-script-for-automation/
- [1]:https://it.wikipedia.org/wiki/Guido_van_Rossum
- [2]:https://gist.github.com/ruimaranhao/4e18cbe3dad6f68040c32ed6709090a3